CURSO BACKEND JAVA & QUARKUS — AULA 05

Módulos de Arquitetura: Config, Client, Constant, Exception e Utils

Uma aplicação backend profissional de alto nível exige desacoplamento e divisão clara de responsabilidades. Nesta aula, exploraremos os 5 pacotes estruturais fundamentais que organizam as configurações, integrações externas, contantes, tratamento global de falhas e funções utilitárias no Quarkus.

1. MÓDULO `CONFIG`: GESTÃO TYPE-SAFE DE PROPRIEDADES

Em vez de espalhar anotações `@ConfigProperty` por todo o sistema, o módulo `config` centraliza a leitura do `application.properties` através de interfaces estritamente tipadas com SmallRye Config.

Recurso do Módulo	Anotação / Classe	Vantagem e Objetivo Prático
Type-Safe Mapping	`@ConfigMapping`	Valida e agrupa propriedades hierárquicas em objetos/interfaces Java no boot.
Valores Padrão	`@WithDefault`	Garante um valor default caso a chave não esteja definida no arquivo de config.
Conversão Automática	`Duration`, `URL`, `Integer`	Converte textos (ex: `"30s"`) diretamente em objetos utilitários de tempo/redes.

1.1. Exemplo Prático: Mapeamento de Configuração da Gateway de Pagamento

Definimos as propriedades no `application.properties` e mapeamos via interface injetável:

```
# application.properties
pagamento.gateway.url=https://api.pagseguro.com/v1
pagamento.gateway.api-key=secrete_key_12345
pagamento.gateway.timeout=15s
pagamento.gateway.max-tentativas=3
```

```
package com.empresa.config;

import io.smallrye.config.ConfigMapping;
import io.smallrye.config.WithDefault;
import java.time.Duration;

@ConfigMapping(prefix = "pagamento.gateway")
public interface GatewayPagamentoConfig {

    String url();

    String apiKey();

    @WithDefault("10s")
    Duration timeout();

    @WithDefault("3")
    int maxTentativas();
}
```

2. MÓDULO `CLIENT`: COMUNICAÇÃO HTTP REATIVA COM MICROPROFILE REST CLIENT

O pacote `client` isola as chamadas HTTP para microsserviços e APIs terceiras. O Quarkus gera dinamicamente a implementação das interfaces anotadas com `@RegisterRestClient`, eliminando o boilerplate de criar instâncias de `HttpClient` manuais.

2.1. Declaração do Cliente de API Externa (Ex: Serviço de CEP)

Mapeamos os métodos e rotas remotas diretamente na interface Java utilizando anotações JAX-RS padrão:

```
package com.empresa.client;

import com.empresa.client.dto.EnderecoViaCepDTO;
import org.eclipse.microprofile.rest.client.inject.RegisterRestClient;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;

@Path("/ws")
@RegisterRestClient(configKey = "viacep-api") // Chave vinculada ao application.properties
@Produces(MediaType.APPLICATION_JSON)
public interface ViaCepClient {

    @GET
    @Path("/{cep}/json")
    EnderecoViaCepDTO buscarEnderecoPorCep(@PathParam("cep") String cep);
}
```

2.2. Vinculação no Properties e Injeção de Dependência no Serviço

No `application.properties` configuramos a URL base, e no serviço injetamos a interface usando a anotação `@RestClient`:

```
# Configuração da URL da API remota
quarkus.rest-client.viacep-api.url=https://viacep.com.br
```

```
package com.empresa.service;

import com.empresa.client.ViaCepClient;
import com.empresa.client.dto.EnderecoViaCepDTO;
import org.eclipse.microprofile.rest.client.inject.RestClient;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;

@ApplicationScoped
public class ConsultaEnderecoService {

    @Inject
    @RestClient // Injeta o proxy dinâmico gerado pelo Quarkus
    ViaCepClient viaCepClient;

    public EnderecoViaCepDTO obterEndereco(String cep) {
        return viaCepClient.buscarEnderecoPorCep(cep);
    }
}
```

3. MÓDULO `CONSTANT`: CENTRALIZAÇÃO DE VALORES GLOBAIS

3.1. Eliminando Magic Strings e Agrupando Códigos do Sistema

O pacote `constant` armazena classes utilitárias de constantes e Enums organizados com metadados para evitar repetição de valores literais dispersos no código fonte.

```
package com.empresa.constant;

public final class SecurityConstants {
    public static final String HEADER_CORRELATION_ID = "X-Correlation-ID";
    public static final String HEADER_AUTHORIZATION = "Authorization";
    public static final String BEARER_PREFIX = "Bearer ";

    private SecurityConstants() {} // Impede instanciação
}
```

```
package com.empresa.constant;

public enum CanalNotificacao {
    EMAIL("E-mail Transacional"),
    SMS("Mensagem de Texto"),
    WHATSAPP("Mensagem de Aplicativo");

    private final String descricao;

    CanalNotificacao(String descricao) {
        this.descricao = descricao;
    }

    public String getDescricao() { return descricao; }
}
```

4. MÓDULO `EXCEPTION`: EXCEÇÕES CUSTOMIZADAS E HANDLER GLOBAL

4.1. Criando Exceções de Negócio e o Padrão ExceptionMapper

Quando uma exceção de negócio é disparada, o Quarkus utiliza uma classe anotada com `@Provider` implementando `ExceptionMapper<T>` para interceptar a falha e construir uma resposta JSON padronizada:

```
package com.empresa.exception;

public class RegraNegocioException extends RuntimeException {
    private final String codigoErro;

    public RegraNegocioException(String codigoErro, String mensagem) {
        super(mensagem);
        this.codigoErro = codigoErro;
    }

    public String getCodigoErro() { return codigoErro; }
}
```

```
package com.empresa.exception.mapper;

import com.empresa.exception.RegraNegocioException;
import com.empresa.dto.ErrorResponseDTO;
import jakarta.ws.rs.core.Response;
import jakarta.ws.rs.ext.ExceptionMapper;
import jakarta.ws.rs.ext.Provider;
import java.time.LocalDateTime;

@Provider // Registra o interceptador de erro global no JAX-RS
public class RegraNegocioExceptionMapper implements ExceptionMapper<RegraNegocioException> {

    @Override
    public Response toResponse(RegraNegocioException ex) {
        var payload = new ErrorResponseDTO(
            ex.getCodigoErro(),
            ex.getMessage(),
            LocalDateTime.now()
        );
```

5. MÓDULO `UTILS`: FUNÇÕES PURAS E UTILITÁRIOS SEM ESTADO

O módulo `utils` contém apenas **funções puras e métodos estáticos**. Classes utilitárias não devem manter estado interno (estateless) e devem ter construtores privados para impedir sua instanciação.

5.1. Exemplo Prático: Utilitário de Sanitize e Validação

```
package com.empresa.utils;

public final class DocumentoUtils {

    private DocumentoUtils() {} // Construtor privado obrigatorio!

    public static String apenasNumeros(String texto) {
        if (texto == null) return null;
        return texto.replaceAll("\\D", "");
    }

    public static boolean isCpfValido(String cpf) {
        String limpo = apenasNumeros(cpf);
        return limpo != null && limpo.length() == 11;
    }
}
```

6. VISÃO HOLÍSTICA: COMO OS 5 MÓDULOS INTERAGEM EM UMA REQUISIÇÃO

Em um fluxo de execução real de um endpoint backend, os cinco módulos atuam de forma orquestrada para garantir segurança, resiliência e clareza de código:

Módulo	Papel na Execução da Requisição HTTP
`config`	Fornece credenciais e timeouts lidos das variáveis de ambiente para os serviços no boot.
`utils`	Limpa e sanitiza os dados de entrada recebidos na Controller (ex: remove pontuação de CPF).
`client`	Efetua a chamada HTTP para o microsserviço parceiro utilizando a URL injetada pelo `config`.
`constant`	Fornece os códigos de erro padronizados e headers HTTP estáticos sem magic strings.
`exception`	Intercepta qualquer falha tratada e devolve uma resposta JSON padronizada (ex: HTTP 422 / 400).

6.1. Estrutura Padrão de Pastas do Projeto Quarkus

```
src/main/java/com/empresa/app/
├── client/        - Interfaces @RegisterRestClient e DTOs de integração externa
├── config/        - Mapeamentos @ConfigMapping e propriedades estritamente tipadas
├── constant/      - Enums do domínio e classes de constantes públicas estáticas
├── exception/     - Exceções personalizadas de negócio e ExceptionMappers
└── utils/         - Helper classes e rotinas de formatação estáticas sem estado
```